

EX No:1b	EDA DATA CLEANING

AIM

To perform exploratory data analysis on the Vehicle Analysis Dataset by examining the dataset structure, dimensions, data types, statistical characteristics, missing values, country-wise and region-wise attack distributions, attack types, correlations among numerical attributes, and exporting the dataset into different file formats.

ALGORITHM

1. Import the required libraries such as kagglehub, os, and pandas.
2. Download the Vehicle Analysis Dataset using KaggleHub.
3. Identify the CSV file available in the downloaded dataset.
4. Load the CSV file into a Pandas DataFrame using latin1 encoding.
5. Display the first five and last five records.
6. Display the dataset information, shape, column names, and data types.
7. Generate descriptive statistical information for numerical attributes.
8. Identify the number of missing values in each column.
9. Find the top 10 countries based on the number of terrorist attacks.
10. Analyze the distribution of attacks across different regions.
11. Analyze the different types of terrorist attacks.
12. Display the unique attack types available in the dataset.
13. Generate the correlation matrix for numerical attributes.
14. Export the dataset into CSV, Excel, and JSON formats.
15. Display a confirmation message after successful file creation.

CODE

```
import kagglehub

# Download latest version

path = kagglehub.dataset_download("nehalbirla/vehicle-dataset-from-cardekho")

print("Path to dataset files:", path)


import os

import pandas as pd

print("Dataset path:", path)

print("\nFiles in dataset:")

print(os.listdir(path))


# Find CSV file automatically
```

```
csv_file = [f for f in os.listdir(path) if f.endswith(".csv")][0]

print("\nCSV file:", csv_file)

# Load dataset

df = pd.read_csv(
    os.path.join(path, csv_file),
    low_memory=False
)

print("\nFirst 5 rows:")
display(df.head())

print("\nLast 5 rows:")
display(df.tail())


print("\nDataset information:")
df.info()

print("\nShape:")
print(df.shape)

print("\nColumns:")
print(df.columns.tolist())


print("\nData types:")
print(df.dtypes)

print("\nStatistical description:")
display(df.describe())

print("\nMissing values:")
display(df.isnull().sum())


print("\nTop 10 car names:")
display(df["Car_Name"].value_counts().head(10))

print("\nFuel types:")
display(df["Fuel_Type"].value_counts())

print("\nSeller types:")
display(df["Seller_Type"].value_counts())
```

```
print("\nTransmission types:")
display(df["Transmission"].value_counts())

print("\nUnique fuel types:")
print(df["Fuel_Type"].unique())
print("\nUnique seller types:")
print(df["Seller_Type"].unique())

print("\nUnique transmission types:")
print(df["Transmission"].unique())
print("\nUnique owner types:")
print(df["Owner"].unique())
print("\nCorrelation:")
display(df.corr(numeric_only=True))

# Excel
df.to_excel("vehicle_analysis.xlsx", index=False)

# JSON
df.to_json("vehicle_analysis.json", orient="records", indent=4)

# CSV
df.to_csv("vehicle_analysis.csv", index=False)
print("\nCSV, Excel and JSON files created successfully.")
```

OUTPUT

First 5 rows:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0

Last 5 rows:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
296	city	2016	9.50	11.6	33988	Diesel	Dealer	Manual	0
297	brio	2015	4.00	5.9	60000	Petrol	Dealer	Manual	0
298	city	2009	3.35	11.0	87934	Petrol	Dealer	Manual	0
299	city	2017	11.50	12.5	9000	Diesel	Dealer	Manual	0
300	brio	2016	5.30	5.9	5464	Petrol	Dealer	Manual	0

Dataset information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 301 entries, 0 to 300

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Car_Name	301 non-null	object
1	Year	301 non-null	int64
2	Selling_Price	301 non-null	float64
3	Present_Price	301 non-null	float64
4	Kms_Driven	301 non-null	int64
5	Fuel_Type	301 non-null	object
6	Seller_Type	301 non-null	object
7	Transmission	301 non-null	object
8	Owner	301 non-null	int64

dtypes: float64(2), int64(3), object(4)

memory usage: 21.3+ KB

Shape:

(301, 9)

Columns:

['Car_Name', 'Year', 'Selling_Price', 'Present_Price', 'Kms_Driven', 'Fuel_Type', 'Seller_Type', 'Transmission', 'Owner']

Data types:

```
Car_Name      object
Year          int64
Selling_Price float64
Present_Price float64
Kms_Driven    int64
Fuel_Type     object
Seller_Type   object
Transmission  object
Owner         int64
dtype: object
```

Statistical description:

	Year	Selling_Price	Present_Price	Kms_Driven	Owner
count	301.000000	301.000000	301.000000	301.000000	301.000000
mean	2013.627907	4.661296	7.628472	36947.205980	0.043189
std	2.891554	5.082812	8.644115	38886.883882	0.247915
min	2003.000000	0.100000	0.320000	500.000000	0.000000
25%	2012.000000	0.900000	1.200000	15000.000000	0.000000
50%	2014.000000	3.600000	6.400000	32000.000000	0.000000
75%	2016.000000	6.000000	9.900000	48767.000000	0.000000
max	2018.000000	35.000000	92.600000	500000.000000	3.000000

Missing values:

	0
Car_Name	0
Year	0
Selling_Price	0
Present_Price	0
Kms_Driven	0
Fuel_Type	0
Seller_Type	0
Transmission	0
Owner	0

dtype: int64

Top 10 car names:

	count
Car_Name	
city	26
corolla altis	16
verna	14
fortuner	11
brio	10

	count
Car_Name	
ciaz	9
i20	9
innova	9
grand i10	8
Royal Enfield Classic 350	7

dtype: int64

Unique fuel types:
['Petrol' 'Diesel' 'CNG']

Unique seller types:
['Dealer' 'Individual']

Unique transmission types:
['Manual' 'Automatic']

Unique owner types:
[0 1 3]

Correlation:

	Year	Selling_Price	Present_Price	Kms_Driven	Owner
Year	1.000000	0.236141	-0.047584	-0.524342	-0.182104
Selling_Price	0.236141	1.000000	0.878983	0.029187	-0.088344
Present_Price	-0.047584	0.878983	1.000000	0.203647	0.008057
Kms_Driven	-0.524342	0.029187	0.203647	1.000000	0.089216
Owner	-0.182104	-0.088344	0.008057	0.089216	1.000000

CSV, Excel and JSON files created successfully.

RESULT